AI MOCK FACTORY

Enterprise AI-Powered API Mock Generation and Contract Testing Platform

Complete Product Architecture, Technical Design and Implementation Specification

Version 1.0 | Proposed Architecture | 11 November 2026

AI Mock Factory

Enterprise AI-Powered API Mock Generation and Contract Testing Platform

Complete Product Architecture, Technical Design and Implementation Specification

Version: 1.0 | **Status:** Proposed Architecture | **Date:** 11 November 2026

Document Control

• **Audience:** Architects, Senior Engineers, Platform Engineering, DevOps, QA Automation, Application Teams and Security Engineering.

• **Primary stack:** React, Java 21, Spring Boot, Python FastAPI, OpenAPI, Pact, WireMock, PostgreSQL, Redis, Elasticsearch, Kafka (when justified), Docker, Kubernetes and GitHub Actions.

• **Scope:** Product requirements, architecture, services, AI workflows, data generation, contracts, APIs, database, deployment, security, testing, operations and roadmap.

Table of Contents

• Executive Summary

• Product Architecture Principles

• Functional Requirements

• High-Level Architecture

• Detailed Microservice Architecture

• Contract Processing Design

• AI Intelligence Architecture

• Business-Aware Data Generation

• Scenario Generation Framework

• WireMock Runtime Architecture

• Database Design

• REST API Design

• Asynchronous Processing

• GitHub Actions Integration

• Docker and Kubernetes Deployment

• Security Architecture

• Observability and Monitoring

• Contract Change Management

• Developer Portal Design

• Testing Strategy

• Scalability and Performance

• Error Handling

• Repository Structure

• Detailed Implementation Plan

• Production Readiness Checklist

• Future Enhancements

• Key Architectural Decisions

• Product Differentiation

• Recommended First Release

• Additional Engineering Decisions and Guardrails

• Glossary

1. Executive Summary

1.1 Product overview

AI Mock Factory is a centralized, self-service enterprise platform that automatically generates API mocks, realistic test data, business scenarios and contract validation tests using AI and API specifications. It reduces manual mock creation and dependency on backend service availability.

Supported inputs:

• OpenAPI 3.x / Swagger.

• Pact consumer contracts.

• AsyncAPI specifications.

• Existing API request/response samples.

• GitHub-hosted API specifications.

The platform analyzes definitions, understands API structures and business semantics, generates mock responses, validates them, and publishes executable mock services.

1.2 Business problem

Enterprise applications depend on Customer Management, Account Management, Payment, Transaction, Notification and Authentication services. When dependent services are unavailable, development and integration testing slow down.

Problems addressed:

• Development teams wait for dependent services.

• Integration testing is delayed.

• QA teams manually create mock responses.

• Mock data is inconsistent between services.

• API contract changes break existing mocks.

• Teams implement mocking differently.

• Large mock collections are difficult to maintain.

• Test environments are expensive when all real services must be deployed.

1.3 Product objectives

• Generate mocks automatically from API contracts.

• Understand API structure and business descriptions.

• Generate realistic, configurable test data.

• Generate positive, negative, boundary and failure scenarios.

• Maintain relationships across multiple APIs.

• Validate every generated response against its contract.

• Support REST and asynchronous APIs.

• Generate executable WireMock configurations.

• Deploy mocks using Docker or Kubernetes.

• Support multiple application teams and environments.

• Integrate with GitHub Actions and enterprise CI/CD.

• Detect contract changes and regenerate affected mocks.

• Maintain version history and auditability.

• Provide a self-service developer portal.

1.4 Non-goals for initial release

• Replacing actual production APIs.

• Automatically deploying unapproved application code.

• Using production PII in generated responses.

• Training foundation models from customer data.

• Guaranteeing complete business-rule discovery from schemas alone.

• Replacing provider-side functional testing.

• Automatically modifying production API implementations.

2. Product Architecture Principles

2.1 Contract-first

API contracts are the authoritative source of truth for API structure. AI must not introduce undocumented fields, invalid response types or unsupported HTTP status codes.

2.2 AI-assisted, deterministic execution

AI performs semantic analysis and scenario generation. Deterministic components perform contract parsing, schema validation, data type validation, referential integrity checks, mock configuration generation, runtime deployment and contract verification.

2.3 Multi-tenant architecture

Each application team has a logical workspace. Isolate resources by organization, team, application, environment, API contract and mock deployment.

2.4 Self-service

Application teams should not require platform engineering intervention for ordinary mock creation.

2.5 Reproducibility

The same contract version, generator configuration, seed and scenario definition should produce reproducible results. Persist generated artifacts as immutable versions.

2.6 Extensibility

Support new protocols, AI providers, mock runtimes, databases and deployment platforms through adapters.

3. Functional Requirements

3.1 Contract management

• Upload OpenAPI YAML and JSON.

• Import contracts from GitHub.

• Import Pact files and AsyncAPI specifications.

• Version and compare contracts.

• Validate contracts and track ownership.

• Support approval workflows and change notifications.

3.2 AI analysis

• Read API descriptions and endpoint operations.

• Identify request/response structures, required/optional properties, enums and field descriptions.

• Identify relationships between APIs.

• Suggest business scenarios and missing scenarios.

• Generate synthetic response examples.

3.3 Mock generation

Generate success responses such as 200, 201 and 204, and error responses such as 400, 401, 403, 404, 409, 422, 429, 500, 502 and 503. Only treat status codes supported by the specification or explicitly configured simulation policy as contract-valid responses. Fault injection may intentionally simulate out-of-contract failures and must be labeled separately.

3.4 Mock management

Create, view, edit, clone, enable, disable, deploy, stop, restart and delete mocks. Export WireMock mappings and view execution history and validation results.

3.5 Contract testing

Support request/response schema validation, status/header/content-type validation, Pact consumer verification, provider compatibility checks, backward compatibility analysis and regression testing.

4. High-Level Architecture

4.1 Components

Component	Technology	Responsibility
Developer Portal	React	User interface
API Gateway	Spring Cloud Gateway	Routing and authentication
Platform Backend	Java 21, Spring Boot	Business operations
Contract Registry	PostgreSQL + object storage	Contract metadata, files and versions
Contract Parser	Java/Python adapter	OpenAPI and Pact parsing
AI Orchestrator	Python FastAPI	AI workflow management
LLM Provider	OpenAI or enterprise model	Semantic understanding
Scenario Generator	Python	Scenario generation
Data Generator	Python	Synthetic data
Validation Engine	OpenAPI / JSON Schema validators	Contract compliance
Mock Generator	Java/Python	WireMock configuration
Mock Runtime	WireMock	Executable API mocks
Cache	Redis	Short-lived caching
Vector Search	Elasticsearch	Semantic retrieval
Event Messaging	Kafka, when justified	Asynchronous processing
Deployment Manager	Docker / Kubernetes API	Mock deployment
CI/CD	GitHub Actions	Automation
Observability	OpenTelemetry, Prometheus, Grafana	Monitoring

4.2 Six logical layers

• **User interaction:** React portal for application registration, contract upload, mock configuration, scenario selection, deployment management and test results.

• **Platform services:** Spring Boot services for authentication, authorization, teams, applications, contract lifecycle, mock lifecycle, deployment lifecycle and audit logging.

• **AI intelligence:** Python services for contract interpretation, semantic extraction, relationship discovery, scenario generation, synthetic data generation and contract change analysis.

• **Validation:** Deterministic OpenAPI, JSON Schema, request/response, referential-integrity, Pact and compatibility checks.

• **Execution:** WireMock for request matching, response delivery, stateful behavior, scenario execution, request recording and statistics.

• **Infrastructure:** Docker, Kubernetes, PostgreSQL, Redis, Kafka, object storage and monitoring.

5. Detailed Microservice Architecture

5.1 API Gateway

Use Spring Cloud Gateway to validate authentication tokens, route API requests, enforce rate limits, validate application access, forward correlation IDs, apply tenant-level authorization and route requests to internal services. Use enterprise OIDC integration such as Okta.

5.2 Application Management Service

Create, update and delete applications; associate applications with teams and contracts; manage environments and application permissions. Example: Payments Portal owned by Payments Engineering, with Development, Integration, QA and Performance environments.

5.3 Contract Registry Service

Receive specifications, parse metadata, store documents, create versions, calculate hashes, track compatibility and notify downstream services. PostgreSQL stores metadata; object storage stores original contract files.

Recommended object path:

`contracts/{tenant}/{application}/{contract}/{version}/openapi.yaml`

5.4 Contract Parser Service

Parse OpenAPI; extract paths, methods, parameters, request bodies, responses, component references, enums, required properties, authentication requirements and examples; generate a normalized intermediate representation. Keep the representation independent of the original format.

5.5 AI Orchestration Service

Use Python FastAPI to coordinate workflows, construct prompts, retrieve contract information, invoke the LLM, validate structured outputs, retry invalid outputs, track model versions and token usage, and store generation metadata. The AI service must not directly deploy mocks. It produces structured artifacts for deterministic validation and deployment.

5.6 Scenario Generation Service

Generate positive, negative, boundary, authentication, business-specific and dependency-failure scenarios; identify uncovered response definitions. Each scenario contains ID, name, description, endpoint, method, inputs, expected status, expected response, dependencies, validation status, generation source and confidence score.

5.7 Synthetic Data Generation Service

Generate realistic data, deterministic identifiers, timestamps, addresses, account details and transactions. Maintain relationships, uniqueness and referential integrity.

5.8 Validation Service

Validate generated responses, request payloads, HTTP status codes, headers, content types, required properties, enums, formats, nested objects, references and scenario configuration. Invalid output must never be deployed.

5.9 Mock Configuration Service

Convert validated scenarios into WireMock mappings, generate response files and request matching rules, package deployment artifacts and version generated configurations.

5.10 Mock Deployment Service

Create Docker/Kubernetes deployments, services, ConfigMaps, ingress resources, resource limits and network policies; register endpoints, monitor health, and stop/delete deployments.

6. Contract Processing Design

6.1 Supported formats

• OpenAPI 3.0.x and 3.1.x initially; Swagger 2.0 may be added later.

• Pact consumer contract files, provider verification, contract versioning and consumer/provider relationships.

• AsyncAPI for future Kafka and RabbitMQ support.

6.2 OpenAPI example

```yaml
openapi: 3.0.3
info:
  title: Customer API
  version: 1.0.0
paths:
  /customers/{customerId}:
    get:
      operationId: getCustomer
      parameters:
        - name: customerId
          in: path
          required: true
          schema:
            type: string
      responses:
        '200':
          description: Customer found
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/Customer'
        '404':
          description: Customer not found
components:
  schemas:
    Customer:
      type: object
      required: [customerId, name, status]
      properties:
        customerId:
          type: string
        name:
          type: string
        email:
          type: string
          format: email
        status:
          type: string
          enum: [ACTIVE, INACTIVE]
```

6.3 Normalized representation

```json
{
  "contractId": "customer-api",
  "version": "1.0.0",
  "operations": [{
    "operationId": "getCustomer",
    "method": "GET",
    "path": "/customers/{customerId}",
    "parameters": [{
      "name": "customerId",
      "in": "path",
      "required": true,
      "type": "string"
    }],
    "responses": [
      {"status": 200, "schema": "Customer"},
      {"status": 404}
    ]
  }]
}
```

7. AI Intelligence Architecture

7.1 AI responsibilities

Use the LLM for semantic understanding: business descriptions, field descriptions, relationship suggestions, realistic values, scenario descriptions, missing scenarios and contract change analysis.

Do not use the LLM as the final authority for schema validation, authorization, deployment infrastructure, tenant isolation or deterministic business enforcement.

7.2 Processing pipeline

- Receive normalized contract.
- Retrieve relevant API descriptions.
- Retrieve application business rules.
- Retrieve approved example data.
- Construct a structured prompt.
- Call the selected LLM.
- Parse structured output.

• Validate AI output structure.

• Validate scenarios against contract constraints.

• Retry or reject invalid output.

• Persist generation metadata.

7.3 Prompt template

```
SYSTEM:
You are an enterprise API contract analysis and mock generation engine.
Generate API scenarios using only the supplied API contract and approved business rules.
Do not invent response properties.
Do not change field data types.
Respect required fields, enum values, formats and constraints.
Generate realistic but synthetic data.
Maintain referential integrity across related entities.
Return structured JSON matching the supplied output schema.
If a business rule is ambiguous, identify the ambiguity rather than assuming a rule.

INPUT:
- API contract
- Endpoint definition
- Business description
- Generation requirements
- Existing approved examples
- Related service metadata
- Desired scenario categories
```

7.4 Structured AI output

```
{
  "scenarios": [{
    "scenarioName": "active_customer",
    "description": "Customer with an active account",
    "statusCode": 200,
    "response": {
      "customerId": "C1001",
      "name": "John Smith",
      "email": "john@example.test",
      "status": "ACTIVE"
    }
  }]
}
```

Parse the AI response into a strict schema. Never execute raw generated code.

8. Business-Aware Data Generation

8.1 Problem and example

A basic mock server generates independent responses. Enterprise mock data must be correlated across services.

Customer API:

```
{"customerId":"C1001","name":"John Smith"}
```

Account API:

```
{"accountId":"A5001","customerId":"C1001","balance":5000}
```

Payment API:

```
{"paymentId":"P9001","accountId":"A5001","amount":100}
```

Identifiers must remain consistent across services.

8.2 Relationship engine

Maintain primary keys, foreign keys, entity relationships, unique constraints, cardinality, shared identifiers and parent-child relationships. Example: Customer 1:N Accounts; Account 1:N Payments; Payment 1:1 Payment Status.

8.3 Relationship metadata

Source entity	Source field	Target entity	Target field
Customer	customerId	Account	customerId
Account	accountId	Payment	accountId
Payment	paymentId	PaymentStatus	paymentId

Relationships must be confirmed through contract definitions, application metadata, explicit user rules or approved AI suggestions. AI-discovered relationships are not authoritative until approved.

8.4 Deterministic synthetic data

Support Faker-based data, seeded reproducibility, UUIDs, sequences, configurable ranges, enum values, date constraints and custom business rules.

Example:

```
from faker import Faker
import random

fake = Faker()

def generate_customer(customer_id):
    return {
        "customerId": customer_id,
        "name": fake.name(),
        "email": fake.email(),
        "status": random.choice(["ACTIVE", "INACTIVE"])
    }
```

Production implementation must use a seeded random generator rather than global random state.

9. Scenario Generation Framework

9.1 Categories

• **Positive:** valid customer, valid account, successful payment, successful transaction.

• **Negative:** invalid customer, missing account, invalid payment, unauthorized request.

• **Boundary:** zero balance, maximum transaction amount, minimum allowed value, empty collections, maximum string length.

• **Failure:** downstream unavailable, request timeout, rate limit exceeded, database unavailable, network failure.

• **Stateful:** payment initiated, pending, completed or failed.

9.2 Priority

Each scenario records business importance, contract coverage, risk level, generation confidence and approval requirement.

Scenario	Priority
Successful payment	Critical
Insufficient funds	Critical
Invalid account	High
Empty transaction list	Medium
Backend timeout	High

9.3 User-defined configuration

Number of scenarios, scenario categories, data volume, business rules, failure simulation percentage, stateful behavior and generation seed.

10. WireMock Runtime Architecture

10.1 Why WireMock

WireMock provides HTTP request matching, response configuration, stateful scenarios, request recording, response templating, Docker deployment and Kubernetes deployment. It is the initial runtime adapter.

10.2 Mapping example

```
{
  "request": {
    "method": "GET",
    "urlPathPattern": "/customers/.*"
  },
  "response": {
    "status": 200,
    "headers": {"Content-Type": "application/json"},
    "jsonBody": {
      "customerId": "C1001",
      "name": "John Smith",
      "status": "ACTIVE"
    }
  }
}
```

10.3 Stateful scenario example

```
{
  "scenarioName": "payment_lifecycle",
  "requiredScenarioState": "Started",
  "newScenarioState": "COMPLETED",
  "request": {
    "method": "GET",
    "url": "/payments/P9001"
  },
  "response": {
    "status": 200,
    "jsonBody": {
      "paymentId": "P9001",
      "status": "COMPLETED"
    }
  }
}
```

Use the exact mapping schema supported by the selected WireMock version.

10.4 Runtime isolation

Each deployment has a unique deployment ID, isolated runtime/namespace, CPU and memory limits, network restrictions, controlled ingress, expiration and health checks.

11. Database Design

11.1 System of record

PostgreSQL is the system of record for platform metadata. Store original contract files and generated artifacts in object storage; retain URIs and checksums in PostgreSQL.

11.2 Core entities

Entity	Important fields
Tenant	tenant_id, tenant_name, status, created_at, updated_at
Team	team_id, tenant_id, team_name, owner, created_at
Application	application_id, tenant_id, team_id, application_name, description, status, created_at
API Contract	contract_id, application_id, contract_name, contract_type, repository_url, status, created_at
Contract Version	contract_version_id, contract_id, version, storage_uri, checksum, parser_version, compatibility_status, created_at
API Operation	operation_id, contract_version_id, operation_name, http_method, endpoint_path, request_schema, response_schema
Scenario	scenario_id, operation_id, scenario_name, scenario_type, expected_status, configuration_uri, validation_status, generation_version, created_at
Mock Deployment	deployment_id, application_id, environment, runtime_type, runtime_url, deployment_status, resource_configuration, created_at, expires_at
Generation Job	job_id, contract_version_id, job_type, job_status, model_name, prompt_version, generation_seed, started_at, completed_at, error_details
Business Relationship	relationship_id, source_application, source_entity, source_field, target_application, target_entity, target_field, relationship_type, approval_status
Audit Event	audit_id, tenant_id, user_id, action, resource_type, resource_id, timestamp, correlation_id

11.3 Constraints and indexes

All tenant-owned records must have tenant ownership. Use foreign keys where appropriate and unique constraints for tenant name within organization, application name within team, contract version within contract and scenario name within operation.

Index tenant_id, application_id, contract_id, deployment_status, job_status and created_at. Use Flyway or Liquibase migrations.

11.4 PostgreSQL DDL

```
CREATE TABLE tenant (
    tenant_id UUID PRIMARY KEY,
    tenant_name VARCHAR(200) NOT NULL,
    status VARCHAR(30) NOT NULL,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE application (
    application_id UUID PRIMARY KEY,
    tenant_id UUID NOT NULL REFERENCES tenant(tenant_id),
    application_name VARCHAR(200) NOT NULL,
    description TEXT,
    status VARCHAR(30) NOT NULL,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    UNIQUE (tenant_id, application_name)
);

CREATE TABLE api_contract (
    contract_id UUID PRIMARY KEY,
    application_id UUID NOT NULL REFERENCES application(application_id),
    contract_name VARCHAR(200) NOT NULL,
    contract_type VARCHAR(30) NOT NULL,
    status VARCHAR(30) NOT NULL,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE contract_version (
    contract_version_id UUID PRIMARY KEY,
    contract_id UUID NOT NULL REFERENCES api_contract(contract_id),
    version VARCHAR(50) NOT NULL,
    storage_uri TEXT NOT NULL,
    checksum VARCHAR(128) NOT NULL,
    compatibility_status VARCHAR(50),
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    UNIQUE(contract_id, version)
);

CREATE TABLE generation_job (
    job_id UUID PRIMARY KEY,
    contract_version_id UUID NOT NULL
        REFERENCES contract_version(contract_version_id),
    job_type VARCHAR(50) NOT NULL,
    job_status VARCHAR(30) NOT NULL,
    model_name VARCHAR(150),
    prompt_version VARCHAR(50),
    generation_seed BIGINT,
    started_at TIMESTAMP,
    completed_at TIMESTAMP,
    error_details JSONB
);

CREATE TABLE mock_deployment (
    deployment_id UUID PRIMARY KEY,
    application_id UUID NOT NULL REFERENCES application(application_id),
    environment VARCHAR(100) NOT NULL,
    runtime_type VARCHAR(30) NOT NULL,
    runtime_url TEXT,
    deployment_status VARCHAR(30) NOT NULL,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    expires_at TIMESTAMP
);
```

Additional production tables are required for scenario definitions, API operations, relationships, users, permissions, audit records and environment configuration. Add tenant ownership and row-level security or equivalent authorization enforcement to all tenant-owned data paths.

12. REST API Design

All platform APIs use `/api/v1`. Define pagination, filtering, sorting, idempotency, error responses, request-size limits, authentication, rate limits and OpenAPI documentation for the platform APIs themselves.

12.1 Application APIs

- `POST /api/v1/applications` — create.
- `GET /api/v1/applications/{applicationId}` — get.
- `GET /api/v1/applications` — list.
- `DELETE /api/v1/applications/{applicationId}` — delete.

Example:

```
{
  "applicationName": "payments-portal",
  "teamId": "team-001",
  "description": "Payments application"
}
```

12.2 Contract APIs

• `POST /api/v1/contracts` — multipart file upload with application ID and contract type.

• `GET /api/v1/contracts?applicationId={id}` — list.

• `GET /api/v1/contracts/{contractId}` — get.

• `GET /api/v1/contracts/{contractId}/versions` — versions.

• `POST /api/v1/contracts/{contractId}/compare` — compare versions.

12.3 Generation APIs

`POST /api/v1/generations`

```
{
  "contractVersionId": "contract-version-001",
  "generationMode": "HYBRID",
  "scenarioTypes": ["POSITIVE", "NEGATIVE", "BOUNDARY", "FAILURE"],
  "scenarioCount": 20,
  "generationSeed": 12345,
  "includeRelationships": true
}
```

Response:

```
{"jobId":"job-001","status":"ACCEPTED"}
```

Also provide:

• `GET /api/v1/generations/{jobId}`

• `GET /api/v1/generations/{jobId}/scenarios`

• `POST /api/v1/generations/{jobId}/regenerate`

12.4 Validation APIs

`POST /api/v1/validations`

```
{"scenarioId":"scenario-001","contractVersionId":"contract-version-001"}
```

Response:

```
{"validationStatus":"PASSED","errors":[],"warnings":[]}
```

12.5 Deployment APIs

`POST /api/v1/deployments`

```
{
  "applicationId": "app-001",
  "environment": "integration",
  "runtimeType": "KUBERNETES",
  "scenarioIds": ["scenario-001", "scenario-002"],
  "ttlHours": 24
}
```

Also provide:

• `GET /api/v1/deployments/{deploymentId}`

• `POST /api/v1/deployments/{deploymentId}/stop`

• `DELETE /api/v1/deployments/{deploymentId}`

12.6 Scenario APIs

• `GET /api/v1/scenarios`

• `PUT /api/v1/scenarios/{scenarioId}`

• `POST /api/v1/scenarios/{scenarioId}/activate`

• `POST /api/v1/scenarios/{scenarioId}/deactivate`

13. Asynchronous Processing

13.1 Processing flow

• Developer submits generation request.

• Platform validates request.

• Platform creates generation job.

• Job is published to Kafka or durable job queue.

• AI orchestrator consumes job.

• Contract metadata is retrieved.

• AI generates scenarios.

• Validation service validates output.

• Results are persisted.

• Job becomes COMPLETED or FAILED.

• Portal receives updated status.

13.2 Job states

ACCEPTED, QUEUED, PROCESSING, VALIDATING, COMPLETED, FAILED, CANCELLED.

13.3 Reliability

Use idempotency keys, retry policies, dead-letter queues, timeouts, duplicate-message detection, correlation IDs, failure recovery and durable job state. Kafka is appropriate when workload and event distribution justify it; a durable database-backed queue is sufficient for a smaller initial deployment.

14. GitHub Actions Integration

14.1 Objective

Automatically generate mocks when a pull request introduces or changes API contracts.

14.2 Workflow

• Pull request created.

• GitHub Actions starts.

• Repository checked out.

• OpenAPI files discovered.

• Contracts uploaded.

• Generation job submitted.

• Scenarios generated.

• Contract validation executed.

• Mock artifacts published.

• Mock runtime deployed.

• Integration tests executed.

• Results attached to pull request.

• Ephemeral resources cleaned up.

14.3 Example workflow

```yaml
name: AI Mock Generation

on:
  pull_request:
    paths:
      - "api-contracts/**"
      - "openapi/**"

jobs:
  generate-mocks:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Generate mocks
        run: |
          curl -X POST \
            "$MOCK_FACTORY_URL/api/v1/contracts" \
            -H "Authorization: Bearer $TOKEN" \
            -F "file=@openapi/customer.yaml"

      - name: Run integration tests
        run: ./gradlew integrationTest
```

Production workflow must obtain short-lived credentials through approved identity federation, poll the generation job, verify validation, retrieve the deployment endpoint, and clean up even when tests fail.

15. Docker and Kubernetes Deployment

15.1 Docker

For local development and smaller integration environments, run platform backend, AI service, PostgreSQL, Redis, WireMock and frontend. Docker Compose provides the development environment.

15.2 Kubernetes

Recommended namespaces:

• `mock-factory-platform`

• `mock-factory-ai`

• `mock-factory-runtime`

• `mock-factory-monitoring`

The runtime namespace isolates mock deployments from platform services.

15.3 Resources

Create Deployment, Service, ConfigMap, secret references, Ingress/gateway route, NetworkPolicy, ResourceQuota or resource limits, and TTL cleanup metadata. Never place API keys or credentials directly in ConfigMaps.

15.4 Cleanup

A scheduled cleanup service identifies expired deployments. Delete resources only after checking ownership, tenant association and deployment state.

16. Security Architecture

16.1 Authentication

Use enterprise OIDC. Support Okta or another enterprise identity provider, OAuth 2.0 and JWT validation.

16.2 Authorization

Role	Permissions
Platform Admin	Platform-wide administration
Tenant Admin	Tenant administration
Team Admin	Team and application management
Developer	Contract and mock management
QA Engineer	Scenario and test management
Viewer	Read-only access

Every request must validate both role permissions and tenant ownership.

16.3 Data security

Encrypt data in transit and at rest; protect model credentials; use HashiCorp Vault or cloud secret managers; prevent production secrets from entering prompts; mask sensitive samples; apply retention policies; audit sensitive actions.

16.4 AI security

Implement prompt-injection protection, structured output enforcement, input/output size limits, model access restrictions, tenant-aware retrieval, no cross-tenant retrieval, sensitive-data filtering, timeout handling and token budgets. Treat AI output as untrusted input.

16.5 Production data restrictions

Sanitize production API samples before generation. Never assume synthetic-looking examples are automatically free of PII.

17. Observability and Monitoring

Use OpenTelemetry for distributed tracing.

17.1 Platform metrics

API request count, API latency, HTTP error rate, authentication failures, job queue depth and job processing duration.

17.2 AI metrics

Model latency, token usage, generation success rate, schema rejection rate, retry count, cost per generation and model timeout rate.

17.3 Mock runtime metrics

Active deployments, requests per second, response latency, HTTP status distribution, container memory/CPU and deployment failure rate.

17.4 Business metrics

Applications onboarded, generated scenarios, contract coverage, time saved per application and reduction in integration environment dependencies.

18. Contract Change Management

18.1 Change detection

• Compare new contract with previous version.

• Identify added endpoints.

• Identify removed endpoints.

• Identify changed properties.

• Identify required-field changes.

• Identify enum changes.

• Identify response changes.

• Calculate compatibility.

• Identify affected scenarios.

• Generate updated mocks.

18.2 Compatibility categories

Backward compatible, potentially breaking, breaking, and requires manual review.

Change	Typical classification
Add optional response field	Backward compatible
Remove response field	Potentially breaking
Change field type	Breaking
Add required request field	Breaking
Add enum value	Consumer-dependent
Remove enum value	Potentially breaking

Compatibility must account for request/response direction, schema composition, nullability, optionality and consumer expectations. Simple field comparison is insufficient.

18.3 Regeneration

Regenerate only affected artifacts. Do not unnecessarily regenerate unrelated APIs. Create a new mock artifact version instead of overwriting an approved deployment.

19. Developer Portal Design

19.1 Dashboard

Show registered applications, contracts, generated scenarios, active deployments, failed generation jobs and validation statistics.

19.2 Application management

Show application details, team ownership, environment configuration, related API contracts and mock deployments.

19.3 Contract management

Show contract name/version, API operations, compatibility status and change history.

19.4 Scenario management

Show scenario name/category, expected response, validation status, business relationships and enable/disable controls.

19.5 Deployment management

Show deployment ID, runtime type, environment, endpoint URL, health, expiration and request statistics.

19.6 AI generation interface

Allow natural-language instructions, e.g. “Generate five active customers, two inactive customers, and three customers with no associated accounts. Ensure account IDs remain consistent across Customer and Account APIs.” Convert the instruction into a structured generation specification.

20. Testing Strategy

20.1 Unit tests

Test contract parsing, schema extraction, data generation, relationship generation, scenario generation, validation and configuration generation.

20.2 Integration tests

Test backend/PostgreSQL, backend/AI service, AI/model provider, AI/validation service, deployment manager/Kubernetes and GitHub Actions/platform APIs.

20.3 Contract tests

Use Pact between internal platform services. Validate generated mock responses against OpenAPI schemas.

20.4 AI evaluation

Maintain a curated benchmark of contracts with expected scenarios. Measure structural validity, scenario coverage, business-rule compliance, referential integrity, reproducibility, unsupported-field generation rate and human approval rate. Prefer deterministic assertions over relying only on LLM-as-judge scoring.

20.5 Performance tests

Test small/large contracts, concurrent generation jobs, hundreds of operations, large scenario counts and concurrent mock deployments. Set targets using representative workloads.

21. Scalability and Performance

• **Backend:** scale horizontally based on API traffic.

• **AI service:** scale based on queue depth, active model calls, provider rate limits, CPU and memory.

• **Mock runtime:** scale independently per application/environment.

• **Database:** use connection pooling, indexing, pagination and partitioning where appropriate.

• **Elasticsearch:** use for semantic retrieval and contract knowledge search when justified, not as authoritative contract registry.

• **Redis:** use for short-lived cache, coordination where appropriate, rate limiting and runtime metadata cache. Do not use as sole durable job store.

22. Error Handling

Standard error:

```
{
  "errorCode": "CONTRACT_VALIDATION_FAILED",
  "message": "Generated response does not match contract",
  "correlationId": "abc-123",
  "details": [{
    "field": "customerId",
    "expected": "string",
    "actual": "number"
  }]
}
```

Error categories:

• INVALID_CONTRACT

• CONTRACT_VERSION_CONFLICT

• AI_GENERATION_FAILED

• AI_TIMEOUT

• INVALID_AI_OUTPUT

• SCHEMA_VALIDATION_FAILED

• RELATIONSHIP_VALIDATION_FAILED

• DEPLOYMENT_FAILED

• RUNTIME_UNAVAILABLE

• UNAUTHORIZED

• TENANT_ACCESS_DENIED

• RESOURCE_LIMIT_EXCEEDED

Do not return internal model prompts, secrets or stack traces to clients.

23. Repository Structure

```
ai-mock-factory/
|-- frontend/
|   |-- src/components/
|   |-- src/pages/
|   |-- src/services/
|   |-- src/hooks/
|   |-- src/models/
|   |-- package.json
|-- platform-services/
|   |-- api-gateway/
|   |-- application-service/
|   |-- contract-registry-service/
|   |-- mock-management-service/
|   |-- deployment-service/
|   |-- audit-service/
|-- ai-services/
|   |-- orchestrator/
|   |-- contract-analyzer/
|   |-- scenario-generator/
|   |-- data-generator/
|   |-- relationship-engine/
|   |-- prompt-templates/
|   |-- evaluators/
|-- validation/
|   |-- openapi-validator/
|   |-- json-schema-validator/
|   |-- pact-validator/
|   |-- compatibility-engine/
|-- mock-runtime/
|   |-- wiremock-adapter/
|   |-- mock-templates/
|   |-- runtime-manager/
|-- shared/
|   |-- api-models/
|   |-- event-schemas/
|   |-- security/
|   |-- logging/
|-- infrastructure/
|   |-- docker/
|   |-- kubernetes/
|   |-- helm/
|   |-- terraform/
|-- database/
|   |-- migrations/
|   |-- seed-data/
|-- github/
|   |-- workflows/
|   |-- reusable-actions/
|-- tests/
|   |-- unit/
|   |-- integration/
|   |-- contract/
|   |-- performance/
|-- documentation/
|   |-- architecture/
|   |-- api/
|   |-- operations/
|   |-- onboarding/
|-- docker-compose.yml
|-- README.md
```

Start as a modular monorepo. Split repositories later if independent release ownership is needed.

24. Detailed Implementation Plan

Phase 1 — Foundation and contract ingestion

Deliverables: repository, React shell, Spring Boot backend, PostgreSQL schema, application registration, contract upload, OpenAPI parser, versioning and normalized representation.

Acceptance:

• Users register applications.

• Users upload OpenAPI contracts.

• Contracts parse and version correctly.

• Invalid contracts return useful errors.

Phase 2 — AI generation

Deliverables: FastAPI service, LLM integration, prompt management, scenario generation, synthetic data generation, structured output and job management.

Acceptance:

• Generate valid JSON scenarios.

• Support positive and negative cases.

• Respect schema constraints.

• Track model/prompt versions.

• Reproduce outputs where configured.

Phase 3 — Validation and runtime

Deliverables: OpenAPI response validation, JSON Schema validation, scenario validation, WireMock generator, Docker runtime and mock lifecycle.

Acceptance:

• Invalid mocks cannot deploy.

• Valid mocks start.

• APIs return expected responses.

• Scenario selection works.

• Runtime health is monitored.

Phase 4 — Enterprise integration

Deliverables: GitHub integration, Kubernetes deployment, RBAC, tenant isolation, audit logging, reusable CI/CD workflow and cleanup.

Acceptance:

• Multiple teams use the platform.

• Cross-tenant access is denied.

• Mocks deploy in isolated environments.

• Pull requests trigger generation and tests.

Phase 5 — Advanced AI intelligence

Deliverables: multi-API relationship discovery, business-aware generation, natural-language scenarios, contract impact analysis, edge-case generation and coverage recommendations.

Acceptance:

• Correlated test data works across services.

• AI recommendations are traceable.

• Human approval is available for inferred rules.

• Contract changes identify impacted artifacts.

25. Production Readiness Checklist

Platform

■ Multi-tenant support

■ Application registration

■ Team ownership

■ RBAC

■ Contract versioning

■ API documentation

■ Audit logging

AI

■ Model provider abstraction

■ Prompt versioning

■ Structured outputs

■ Token usage monitoring

■ Retry policy

■ Hallucination safeguards

■ Evaluation benchmark

■ Sensitive-data filtering

Generation

■ Positive scenarios

■ Negative scenarios

■ Boundary scenarios

■ Failure scenarios

■ Synthetic data

■ Relationship integrity

■ Reproducible generation

Validation

■ OpenAPI validation

■ JSON Schema validation

■ Request validation

■ Response validation

■ Pact verification

■ Compatibility checks

Runtime

■ WireMock integration

■ Docker deployment

■ Kubernetes deployment

■ Runtime health checks

■ TTL cleanup

■ Resource limits

■ Network isolation

DevOps

■ GitHub Actions integration

■ CI/CD automation

■ Container scanning

■ Secret management

■ Deployment rollback

■ Monitoring

■ Disaster recovery

26. Future Enhancements

• AsyncAPI and Kafka mock generation.

• GraphQL mock generation.

• gRPC mock generation.

• AI-generated database fixtures.

• Data-driven mock scenarios.

• Sanitized production traffic replay.

• Automated mock regeneration on contract changes.

• Automatic pull requests with updated mocks.

• API dependency graph visualization.

• AI-based test coverage recommendations.

• Model selection based on generation task.

• Cost optimization and model routing.

• Self-service ephemeral test environments.

• Multi-cloud deployment.

• Enterprise mock marketplace.

27. Key Architectural Decisions

Decision	Selected approach	Reason
Frontend	React	Developer portal
Platform backend	Java 21 / Spring Boot	Enterprise services
AI backend	Python / FastAPI	AI ecosystem
Contract format	OpenAPI	Contract-first foundation
Contract testing	Pact	Consumer-driven testing
Mock engine	WireMock	Programmable HTTP simulation
Database	PostgreSQL	Relational metadata
Vector search	Elasticsearch	Semantic retrieval
Cache	Redis	Short-lived caching
Messaging	Kafka when needed	Asynchronous processing
Runtime	Docker and Kubernetes	Flexible deployment
CI/CD	GitHub Actions	Automated integration
Security	OIDC + RBAC	Enterprise access control
AI generation	Hybrid	Semantic flexibility and deterministic reliability

28. Product Differentiation

AI Mock Factory should not position itself as a replacement for WireMock, Postman or Pact. Those tools already provide API development, simulation and testing capabilities. The product differentiates through orchestration.

• **Contract intelligence:** understand API contracts and business semantics.

• **Multi-service intelligence:** understand relationships across microservices.

• **Correlated synthetic data:** generate consistent data across service boundaries.

• **AI scenario generation:** identify and create meaningful test cases.

• **Contract compliance:** validate every generated response before deployment.

• **Automated delivery:** publish mocks into Docker, Kubernetes and ephemeral environments.

• **Self-service:** let application teams independently generate and manage mock environments.

Product promise: “Give AI Mock Factory your API contracts, and it will generate, validate and deploy a complete mock environment for your application.”

29. Recommended First Release Scope

Do not implement every advanced feature in the first release. The first usable release should support:

• OpenAPI 3.x.

• Multiple applications and teams.

• Manual contract upload.

• GitHub contract retrieval.

• AI-assisted scenario generation.

• Deterministic synthetic data.

• JSON Schema validation.

• WireMock generation.

• Docker deployment.

• Basic Kubernetes deployment.

• GitHub Actions integration.

• Scenario management.

• Audit logging.

The first release must prove that a developer can upload a contract, generate scenarios, validate them and obtain a running mock API without manually writing mock configurations. This is the foundation of the enterprise product.

30. Additional Engineering Decisions and Implementation Guardrails

30.1 Contract validity versus simulated failures

A simulated 500 or 503 response may intentionally be outside the normal success schema. Model these as explicit fault-injection scenarios with separate validation rules. Do not mark every simulated failure as a schema-valid business response.

30.2 Schema dialect and reference handling

The parser must resolve local and permitted external references safely, support OpenAPI 3.0 and 3.1 schema differences, preserve nullable/optional semantics, handle allOf/oneOf/anyOf, and prevent untrusted remote-reference retrieval from becoming an SSRF vector.

30.3 Deterministic generation

Persist generator version, Faker/library version, seed, locale, timezone, prompt version, model identifier, normalized contract checksum and generation configuration. Reproducibility across different model versions cannot be assumed; store generated artifacts as immutable versions.

30.4 Artifact lifecycle

Generated artifacts should move through DRAFT, VALIDATED, APPROVED, PUBLISHED, DEPRECATED and ARCHIVED states. Shared production-like environments should require approval before publishing.

30.5 Multi-service deployment

Represent an application mock environment as a deployment bundle containing multiple mock services, their dependency graph, configuration, shared synthetic dataset, network topology and versioned artifacts. Deploy atomically where possible, or expose partial deployment state and rollback.

30.6 Operational policies

Define quotas for active deployments, CPU, memory, generation tokens, contract size, scenario count and runtime duration. Apply per-tenant limits and provide clear quota errors.

30.7 Licensing and third-party compliance

Review licenses, commercial terms, data-processing terms and security posture for WireMock editions, Pact tooling, LLM providers, schema validators and all transitive dependencies before enterprise adoption.

31. Glossary

Term	Meaning
API contract	Machine-readable agreement describing API operations, inputs, outputs and constraints
OpenAPI	Specification format for describing HTTP APIs
Pact	Consumer-driven contract testing framework and contract format
AsyncAPI	Specification for event-driven and message-based APIs
Mock	Simulated service that returns configured responses
Scenario	Named set of request conditions and expected simulated behavior
WireMock mapping	Configuration describing request matching and response behavior
Synthetic data	Artificially generated data that need not represent real individuals
Referential integrity	Consistency of identifiers and relationships across related data
Tenant	Organization or isolated customer boundary within the platform
Ephemeral environment	Temporary environment created for a test or pull request and later destroyed
Contract compatibility	Assessment of whether a contract change may break existing consumers

Document Completion Note

This specification covers product vision, functional requirements, architecture, service responsibilities, contract processing, AI orchestration, data generation, scenario design, mock runtime, database entities and sample DDL, platform REST APIs, asynchronous processing, GitHub Actions, Docker and Kubernetes, security, observability, contract change management, portal design, testing, scalability, error handling, repository structure, implementation roadmap, production readiness, future enhancements, key decisions, differentiation, first-release scope, engineering guardrails and glossary.